Parallelism in Rust

Agenda

- Inherited Mutability
- Interior Mutability, RefCell
- std::thread, Send, Sync
- Arc, Mutex
- Quiz

Safe Rust

What does safe Rust give us?

- No dangling / null references
- No segfaults
- No use-after-free
- No data races
- Type safety

How?

External / Inherited Mutability in Rust

- One writer, or multiple readers, but not both
  - Mutability XOR Aliasing
  - Enforced at compile time through the borrow checker

External / Inherited Mutability in Rust

```
struct Point { x: u32, y: u32 }

// the variable is mutable...
let mut p = Point { x: 10, y: 20 };
// ...and so are fields reachable through this variable
p.x = 11;
p.y = 22;

let q = Point { x: 10, y: 20 };
q.x = 33;  // compilation error
```

External / Inherited Mutability in Rust

```
{
    let px: &u32 = &p.x;  // okay
}
{
    let py: &mut u32 = &mut p.x;  // okay, because p is mut
}
{
    let qx: &u32 = &q.x;  // okay
}
{
    let qy: &mut u32 = &mut q.y;  // compilation error since q is
not mut
}
```

Sharing ownership with Rc<T>

```
{
    let x1: Rc<u32> = Rc::new(1);
    let x2: Rc<u32> = x1.clone();  // create another
reference to the same data
    let x3: Rc<u32> = x2.clone();  // even another
}
// here all references are destroyed and the memory they
were pointing at is deallocated
```

Use case example: Nodes in a tree want to share a reference to a DB

What if every node, on access, wants to update an access count in the DB?

Example: Sharing a Database

```
struct Node {
    db: Rc<Database>,
    id: usize,
}

fn on_access(&mut self) {
    self.db.accesses[self.id] += 1;
}
```

But we “know” that only one node will be modifying the Database at a time…

Does not compile!
Rc only gives you an immutable reference

Example: Sharing a Database

What if we *really* want two writers, but we *promise* they won't write at the same time?

- In other words, check borrow rules at runtime instead of at compile time

RefCell<T>

- Holds an item of type T
- With a shared reference &refcell, you can obtain a shared reference to the item T iff no one else currently has a mutable reference
- With a shared reference &refcell, you can obtain a mutable reference to the item T iff no one else currently has a mutable reference AND no one has a shared reference
- Else, panic!
- Borrow checking rules implemented at runtime!
  - (RefCell only works for single-threaded code)

RefCell Example

```
struct Node {
    db: Rc<RefCell<Database>>
    id: usize,
}

fn on_access(&mut self) {
    self.db.borrow_mut().accesses[self.id] += 1;
}
```

Works even if &self is not mut!

Through a &self, we get a &mut Database!

Interior Mutability

Going from a &T1<T2> to a &mut T2

- should be impossible using standard Rust!
- it is! the impl of RefCell uses unsafe!

Who cares about single threaded code?

Let’s write threads in Rust!

std::thread

- thread::spawn takes in a closure and spawns a new thread running it
  - Returns a JoinHandle which can be later .join()'d

std::thread Example

```
let handle = thread::spawn(|| {
    println!("Thread 2");
});

println!("Thread 1");

handle.join();
```

std::thread Example

```
let mut my_counter = 0;

let handle = thread::spawn(|| {
    my_counter += 1;
    println!("Thread 2");
});
my_counter += 1;
println!("Thread 1");

handle.join();
```

std::thread Example

```
let my_counter = 123;

let handle = thread::spawn(|| {

    println!("Thread 2 reads {my_counter}");
});

println!("Thread 1 reads {my_counter}");

handle.join();
```

std::thread Reality

- Threads cannot share immutable references
  - (Unless they're globals, which have a 'static lifetime)
  - What if we use Rc?

std::thread Example

```
let my_counter = Rc::new(123);
let my_counter_2 = my_counter.clone();;
let handle = thread::spawn(move || {

    println!("Thread 2 reads {my_counter
});

println!("Thread 1 reads {my_counter}");

handle.join();
```

Error: cannot “Send” an Rc

Solution: use the thread-safe Arc (atomic reference count)

Arc: Atomic Reference Counting

- Arc<T> is an Rc<T> but with an atomic counter instead of a normal i32
  - Atomic counters provide shared immutable access between threads!
  - Safe to share an Arc<()>

std::thread Example

```
let my_counter = Arc::new(123);
let my_counter_2 = my_counter.clone();;
let handle = thread::spawn(move || {

    println!("Thread 2 reads {my_counter_2}");
});

println!("Thread 1 reads {my_counter}");

handle.join();
```

std::thread Reality

- Threads cannot share mutable references
  - They can share immutable references using Arc or globals
  - Hmm, if only there was a way to get a mutable reference from a shared one…

std::thread Thoughts?

```
let my_counter = Arc::new(RefCell::new(0_i32));
let my_counter_2 = my_counter.clone();
let handle = thread::spawn(move || {
    my_counter_2.borrow_mut() += 1;
    println!("Thread 2");
});
my_counter.borrow_mut() += 1;
println!("Thread 1");

handle.join();
```

Two threads borrow_mut -> panic!
Data race! Unsynchronized RMW!
Rust will not compile this

std::thread Thoughts?

```
let my_counter = Arc::new(RefCell::new(0_i32));
let my_counter_2 = my_counter.clone();
let handle = thread::spawn(move || {
    my_counter_2.borrow_mut() += 1;
    println!("Thread 2");
});
my_counter.borrow_mut() += 1;
println!("Thread 1");

handle.join();
```

RefCell is not “Sync”
Cannot be shared across threads

Marker traits Send & Sync

If T: Send, then t can be sent (ownership transferred) across threads

If T: Sync, then &t can be sent across threads

- &T: Send iff T: Sync

Generally automatically derivable

RefCell is Send, but not Sync

Why?

Giving it to a new thread is fine.

Sharing it with a new thread implies a data race on the internal shared & mutable active borrow counters

Rc isn’t Send or Sync :(

- You can’t pass in a &Rc<T> to a thread. Why not?
  - Multiple threads mutating internal reference counter without synchronization -> data race
- You can’t pass in a Rc<T> to a thread. Why not?
  - Clones of it could exist in different threads
- Thus, Rc is not Sync or Send
- We need a variant that is!

Arc: Atomic Reference Counting

- Arc<T> is Send and Sync iff T is Send and Sync
  - Arc<RefCell<T>> still has a data race in the internal RefCell access counter
  - We need a type that provides interior mutability (&T1<T2> -> &mut T2) that is both Send & Sync!

Mutex: Thread-Safe Interior Mutability

Mutex<T> “holds” a value T and uses a Mutex to guard accesses to T to one at a time

- different than C, where a Mutex and it’s locked data are unrelated in code

mutex.lock() gives you a mutable reference to T, dropping that will release the lock

Thread safe! Because that is the entire purpose of a Mutex

Is Mutex<T> Sync?

Is Mutex<T> Send?

Answer: Both iff T is Send

Mutex Usage

Make a Mutex<u32>, share a reference to it in two threads

- How? Use Arc to make two references to it!
- Arc<Mutex<T>> is the answer to sharing mutable state between two threads

std::thread Example, fixed*

```
let my_counter = Arc::new(Mutex::new(0));
let my_counter_ref2 = my_counter.clone();

let handle = thread::spawn(move || {
    let locked_cntr_1 = my_counter_ref2.lock();
    *locked_cntr_1 += 1;
    // lock is dropped here when scope ends
});
{
    let locked_cntr_0 = my_counter.lock();
    *locked_cntr_0 += 1;
}
handle.join();
```

Arc<Mutex<T>>: Rust’s answer for shared mutable state

Putting it all together:

- Arc<Mutex<T>> is Send + Sync iff T is Send
- Each thread can get a shared reference to the Mutex<T> via the Arc
- Each thread can get a mutable reference to T via the Mutex
- Synchronization types (like Mutex) “hold” data, enforcing their usage to access the data

Mutex Interface

mutex.lock() gives you a MutexGuard

MutexGuard’s drop impl unlocks the lock

If you use MutexGuard in an expression, it drops instantly

Dropping instantly

```
let my_counter = Arc::new(Mutex::new(0));
let my_counter_ref2 = my_counter.clone();

let handle = thread::spawn(move || {
    *my_counter_ref2.lock() += 1;
    // lock is dropped here because it is never bound to a
var
   other_stuff();
});
*my_counter.lock() += 1;
handle.join();
```

Mutex Poisoning

What if a thread panics while holding a lock?

Turns out, the panic escape hatch will still drop all variables, releasing the lock!

- However, your data may be in an inconsistent state
- This is known as a “poisoned mutex”

Mutex Poisoning

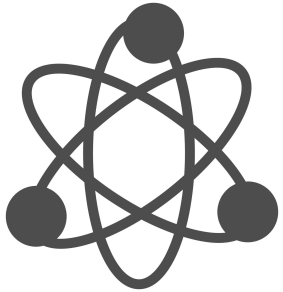
Mutex<T>.lock() actually returns a Result<MutexGuard<T>, PoisonError<T>>

Real applications can detect and recover from poisoning

In our homeworks, we can assume that threads never panic period,

so the general pattern is to use `let guard = mu.lock().unwrap()`

Atomics in Rust



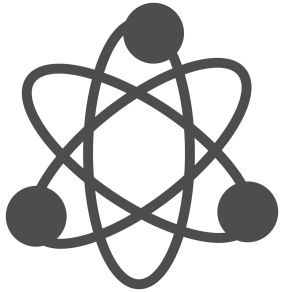
AtomicI32, AtomicUsize, etc…

Each provide load, store, compare_exchange (compare_and_swap), fetch_add…

Question: do you think these types implement interior mutability?

- Does .store require a &mut self or a &self?

Atomics in Rust



Atomics are the foundation for SpinLock, Arc, etc. so they must provide interior mutability! (Two threads can each hold an &atomic)

```
fn fetch_add(&self, addend: i32);
```

Why is this safe?

- Data races are not possible with Atomics!
- Race conditions are, but they’re application bugs that are always possible

Other Synchronization Primitives in Rust

- RwLock: Threadsafe version of RefCell
  - One writer can have the lock (getting a &mut), or many readers can (getting a &)
- SpinLock: not in std, but very possible to make
- Barrier
- CondVar
- Channel (“mpmc”: multi producer, multi consumer)
- Once*: useful stuff for one-time initialization of globals

Quiz on Gradescope (11:14-11:20)

Quiz on Gradescope (11:14-11:20)

A. In a LinkedList, we use Box instead of & because we want ownership of the next node.
B. std::mem primitives (like swap) are required in a linked list push front.
C. Closures move ownership of variables into their scope by default, rather than borrowing them.
D. If a trait has an associated type, then structs that implement that trait provide a type alias for that associated type, binding it to a real type.
E. If you have a `&dyn Animal` , it determines which `say_hi` method to call at compile time.